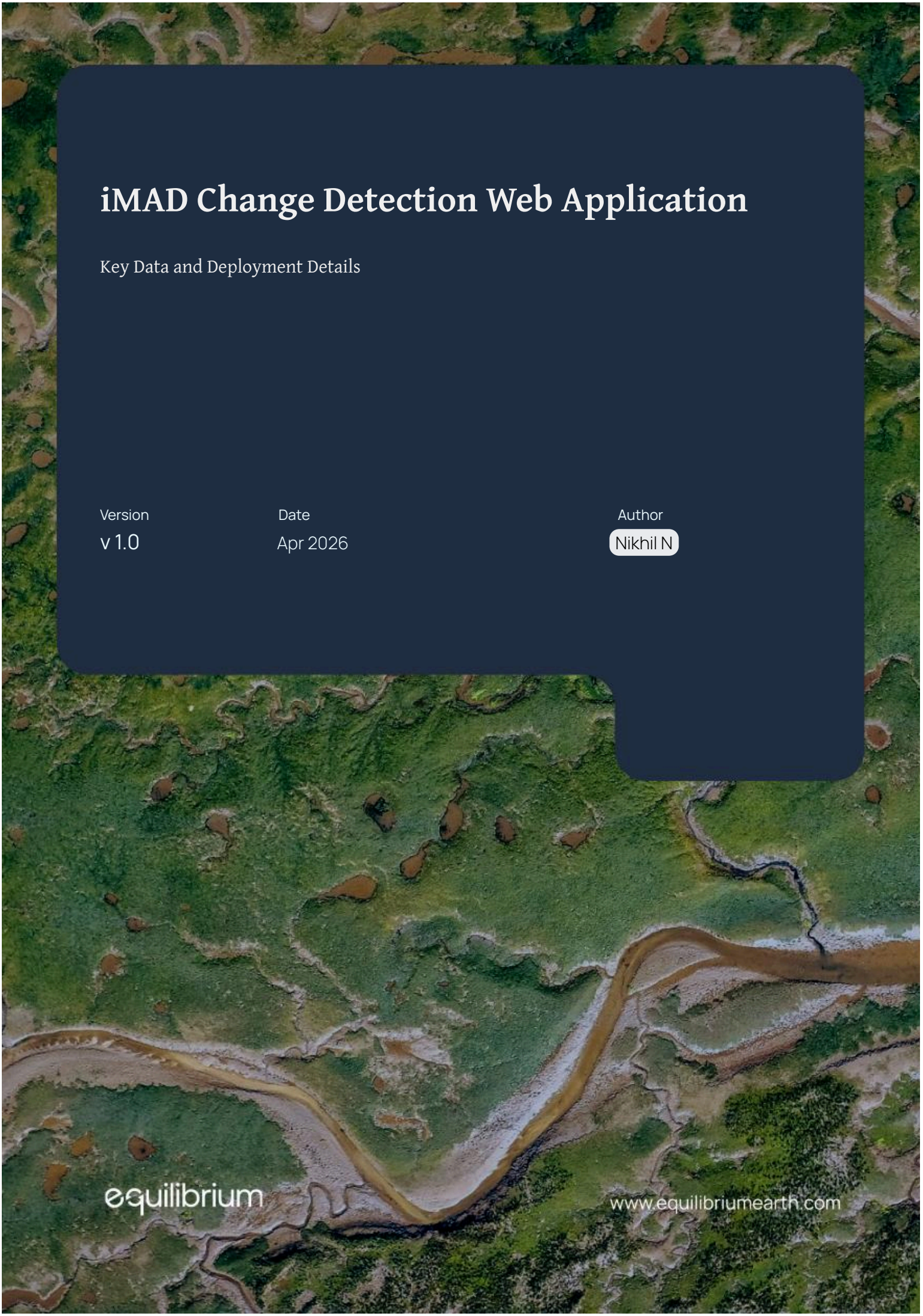
iMAD Change Detection Web Application

Key Data and Deployment Details

Version	Date	Author
v 1.0	Apr 2026	Nikhil N



equilibrium

www.equilibriumearth.com

Table of Contents

iMAD Change Detection Web Application 1
- Function 3
- Data Storing 3
- Deployment: 4
- Capacity & Limitations of the Deployment 6
- Key Risks 6

Function

This is a forest disturbance monitoring tool built only for companies **Equilibrium (EQ)** and **Green Collar (GC).** It detects land cover changes between two dates using Sentinel-2 satellite imagery and the iMAD (Iteratively re-weighted Multivariate Alteration Detection) algorithm supporting GC's carbon farming projects across three states of Australia (NSW, QLD and WA).

Data Storing:

There are **three** distinct data layers, and **none of it is stored in a database:**

1. **Project Boundary Data** - **Change Detection Tracker.xlsx**
   Local on the server A file called **Change Detection Tracker.xlsx** sits alongside main script **app.py** in the deployment container here . It contains a table of project names, IDs, project codes, and ERF registered links to project boundaries stored on the server. When a user picks a project from the dropdown, it reads the boundary directly from that Excel sheet.

Project ID	Project Code	Project name	ERF ID	Boundary
500300	ABB	Abbieglassie	ERF103307	https://cer.gov.au/sites/default/files/projectFiles/ERF103307.kml
609600	ALD	Aldville	ERF111058	https://cer.gov.au/sites/default/files/projectFiles/ERF111058.kml
618041	ALP	Alpana Station	ERF186735	https://cer.gov.au/sites/default/files/projectFiles/ERF186735.kml
501149	ALU	Alum Rock	ERF170462	https://cer.gov.au/sites/default/files/projectFiles/ERF170462.kml
140037	ARA	Arabella	ERF101475	https://cer.gov.au/sites/default/files/projectFiles/ERF101475.kml
602400	ARB	Arbon	EOP100275	https://cer.gov.au/sites/default/files/projectFiles/EOP100275.kml
307200	ARG	Argyle	EOP101086	https://cer.gov.au/sites/default/files/projectFiles/EOP101086.kml
304000	ASH	Ashwood	EOP100580	https://cer.gov.au/sites/default/files/projectFiles/EOP100580.kml

2. **Satellite Imagery** - **Microsoft Planetary Computer (Azure cloud)**

   No satellite data is stored locally. The app queries the Microsoft Planetary Computer STAC API (https://planetarycomputer.microsoft.com/api/stac/v1) in real-time. It fetches Sentinel-2 L2A tiles on-demand, streams them over the network, and processes them in RAM (via stackstac + xarray). The imagery lives in Microsoft Azure Blob Storage - our app just accesses it.

3. **Results** - **Ephemeral (in-memory + temporary files)**

   All computed outputs (iMAD rasters, disturbance shapefiles, masked images) exist **only in memory during the session**. When you click "Download," a ZIP is assembled in a system tempfile directory (deleted afterward). There is no persistent output storage - ***if you close the session before downloading, results are lost.***

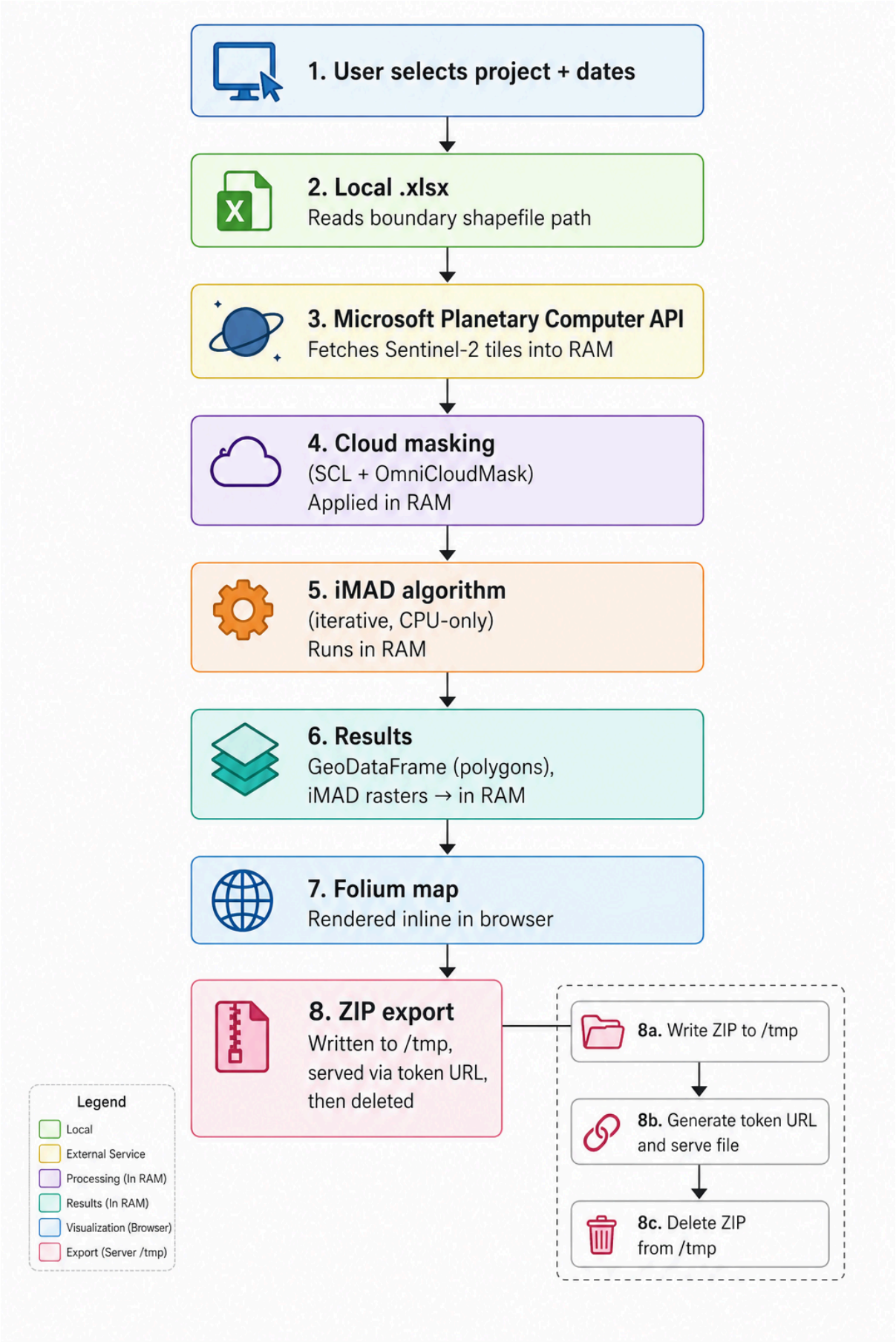
Deployment:

Platform: **Hugging Face Spaces**

The app runs on https://huggingface.co/spaces/nikhil-eq/iMAD-MultiPage, which is a **cloud-hosted** container managed by Hugging Face.

Framework: Solara (a Python reactive web framework, similar to Streamlit but built on Voila/Jupyter). It runs as a Python web server (solara run app.py), with a **Starlette/ASGI backend.**

Authentication: Auth0 via solara-enterprise The app uses solara_enterprise.auth for login. Only email addresses from @equilibriumearth.com or @greencollargroup.com.au can log in.


1. User selects project + dates
2. Local .xlsx
Reads boundary shapefile path
3. Microsoft Planetary Computer API
Fetches Sentinel-2 tiles into RAM
4. Cloud masking
(SCL + OmniCloudMask)
Applied in RAM
5. iMAD algorithm
(iterative, CPU-only)
Runs in RAM
6. Results
GeoDataFrame (polygons),
iMAD rasters → in RAM
7. Folium map
Rendered inline in browser
8. ZIP export
Written to /tmp,
served via token URL,
then deleted
8a. Write ZIP to /tmp
8b. Generate token URL and serve file
8c. Delete ZIP from /tmp
Legend
Local
External Service
Processing (In RAM)
Results (In RAM)
Visualization (Browser)
Export (Server /tmp)

Capacity & Limitations of the Deployment

Aspect	Detail
CPU	Free/basic Hugging Face Space = 2 vCPUs (upgradeable to 8 with paid tier)
RAM	Free tier = ~16 GB; this is critical because the app loads entire Sentinel-2 rasters into memory
Disk	Ephemeral - resets on restart. The xlsx + boundary shapefiles must be bundled in the Space repo
Concurrency	Single-server Solara - multiple users share the same Python process. Heavy concurrent runs will contend for CPU/RAdM
Processing time	iMAD runs up to 100 iterations per pixel; a large ROI can take 5 to 15+ minutes
No GPU	All computation is CPU-only NumPy/SciPy - no GPU acceleration
Session persistence	Zero - results exist only during the active browser session
No database	No SQL, no object store, no write-back of results anywhere
Network dependency	Entirely dependent on Microsoft Planetary Computer API being reachable. Slow/unavailable STAC API = broken app
Image size limits	Very large ROIs (e.g., 100+ km²) will exhaust RAM on the free tier since imagery is loaded as full NumPy arrays

Key Risks:

- **Bigger Region**: Projects with multiple scenes (WA) exceed available RAM limit, need to upgrade to pro version for better performance.
- No output persistence - **every run must be downloaded immediately.**
- Single-user performance - if two people run it simultaneously on a free Space, they'll **compete for RAM and likely cause OOM error**